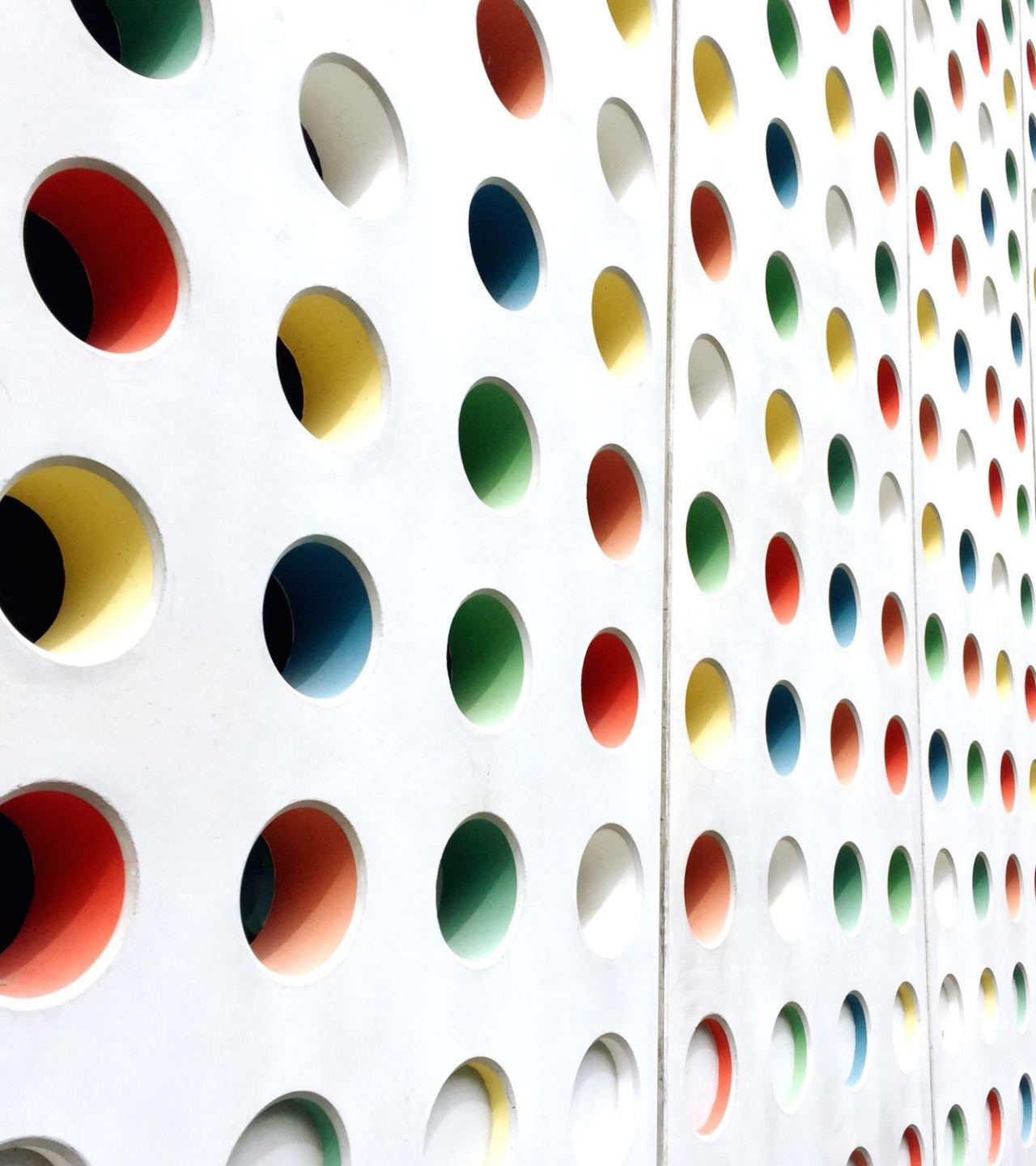




# Semáforos en Java

## PRÁCTICA 3: SEMÁFOROS

---

JUAN ANTONIO RODRIGO FERRÁN

[[JUANANTONIO.RODRIGO@UPM.ES](mailto:JUANANTONIO.RODRIGO@UPM.ES)]

# Índice

---

1. Introducción a los semáforos
2. Semáforos en Java
3. Ejercicios
4. Ejercicio 1 (prioridad de hilos)
5. Ejercicio 2 (acceso a recursos)
6. Ejercicio 3 (ejecución ordenada)
7. Ejercicio 4 (productor-consumidor)

# 1. Introducción a los semáforos

---

❖ En entornos concurrentes pueden surgir distintos problemas:

❖ **Condiciones de carrera:** Dos hilos acceden y modifican un recurso al mismo tiempo

❖ **Bloqueos (Deadlocks):** Dos o más hilos se quedan bloqueados esperando recursos

❖ **Hambruna (Starvation):** Uno o más hilos tienen dificultades para acceder a un recurso en comparación con el resto de los hilos, lo que provoca retrasos o bloqueos en su acceso

❖ Herramientas de sincronización para la resolución de problemas:

❖ Semáforos, mutex, monitores, locks...

❖ Tipos de semáforos:

❖ **Contador:** para controlar el número de elementos que acceden a un recurso

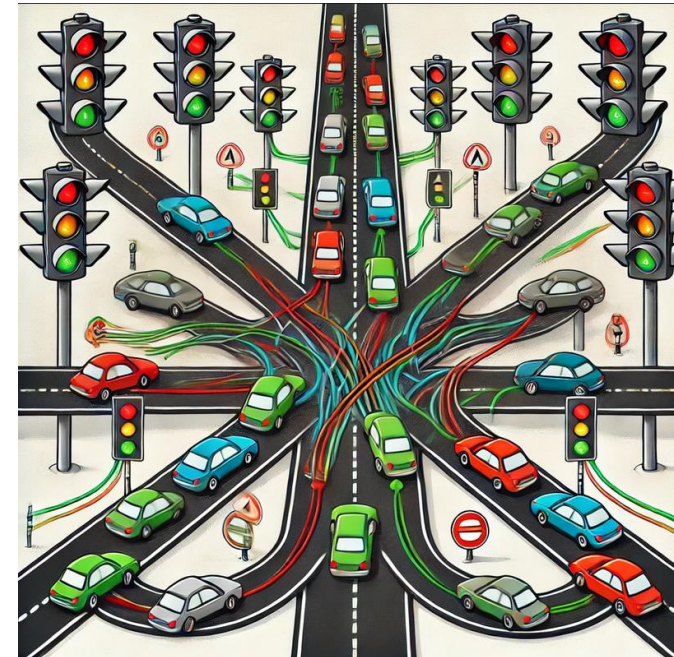
❖ **Binario:** equivalente a un mutex porque sólo puede acceder al recurso un único elemento a la vez

# 1. Introducción a los semáforos

---

## ❖ Usos de semáforos:

- ❖ Controlar el orden de ejecución de hilos
- ❖ Controlar el acceso a un recurso
- ❖ Controlar los elementos disponibles de un recurso



## 2. Semáforos en Java

---

❖ `Java.concurrent.semaphore`

❖ Constructores:

❖ **Semaphore (int permits):** indica el número inicial de permisos (el número de elementos que pueden tener acceso a la vez). Se modifica según se llame a *acquire()* y *release()*.

❖ **Semaphore (int permits, boolean fair):** define si el semáforo es justo

❖ **True**=> Los hilos se atienden en orden de llegada

❖ **False**=> Puede haber acceso desordenado

❖ Métodos principales:

❖ **acquire()/acquire(int n):** disminuye el contador de permisos del semáforo. Si no hay permisos, el hilo queda bloqueado hasta que haya uno disponible. Con `int n`, intentará conseguir `n` permisos a la vez.

❖ **release()/release(int n):** aumenta el contador de permisos de semáforo

## 2. Semáforos en Java

---

### ❖ Otros métodos:

- ❖ **tryAcquire()/tryAcquire(int n, long timeout, TimeUnit unit):** intenta conseguir un permiso sin bloquear; devuelve *true* si tiene éxito, *false* si no. Además permite obtener n permisos en un plazo limitado antes de desistir.
- ❖ **availablePermits():** devuelve el número actual de permisos disponibles
- ❖ **drainPermits():** devuelve y reduce a 0 todos los permisos disponibles

# 3. Ejercicios

---

- ❖ Esta sesión práctica NO será evaluable
- ❖ Como en el caso de la práctica 1, esta práctica servirá como preparación para la práctica 4, que SERÁ evaluada
- ❖ Para realizar esta práctica es necesario descargar o clonar el código de Gitlab (enlace en Moodle)

## 4. Ejercicio 1 (prioridad de hilos)

---

- ❖ El archivo de la clase Semaforo1.java contiene la clase a utilizar en este primer ejercicio
- ❖ Para ejecutarlo se puede usar la clase Main.java, donde se creará un objeto de la clase Semaforo1 y se ejecutará el método que lanza los hilos
- ❖ La clase Semaforo1 utiliza un semáforo para forzar que uno de los hilos se ejecute antes que el otro
- ❖ Ejecuta la clase una vez, modifica la creación del semáforo (con *permits*=1) y compara los resultados



## 5. Ejercicio 2 (acceso a recursos)

---

- ❖ El archivo de la clase Semaforo2.java contiene la clase a utilizar en este segundo ejercicio
- ❖ La clase Semaforo2 utiliza un semáforo para dar acceso de ejecución de un hilo de forma excluyente, con el fin de resolver el problema de la Sección Crítica (Mutex)
- ❖ En este caso no se fija la prioridad, ambos hilos pueden llegar a ejecutarse primero
- ❖ Completa el código que falta en la clase
- ❖ Ejecuta la clase modificando la creación del semáforo (con *permits* = 0, 1 y 2) y compara los resultados
- ❖ Añade un tercer hilo, ejecuta con *permits*=2 y analiza los resultados

## 6. Ejercicio 3 (ejecución ordenada)

---

- ❖ El archivo de la clase Semaforo3.java contiene la clase que se va a utilizar en este tercer ejercicio
- ❖ La clase Semaforo3 utiliza tres semáforos para dar acceso a la ejecución a tres hilos de forma ordenada
- ❖ Rellena el código que falta en la clase y crea los semáforos correctamente para que los hilos se ejecuten en orden creciente y cíclicamente (1, 2, 3, 1, 2, 3...)
- ❖ Analiza si una inicialización incorrecta de los semáforos podría provocar una situación de bloqueo
- ❖ ¿Se podría realizar la misma tarea con dos semáforos si no fuera cíclica?

# 7. Ejercicio 4 (productor-consumidor)

---

- ❖ El archivo de clases `ProductorConsumidor.java` contiene la clase a usar en este tercer ejercicio
- ❖ La clase `ProductorConsumidor` utiliza tres semáforos para controlar el acceso a un búfer que almacena hasta 5 números
  - ❖ Un semáforo se ocupa de garantizar el acceso al búfer de forma exclusiva para cada hilo
  - ❖ Un semáforo se encarga de permitir el acceso de escritura si queda sitio
  - ❖ Un semáforo se encarga de permitir el acceso de lectura si no está vacío
- ❖ Los números son almacenados por dos productores
- ❖ Los números son retirados por dos consumidores

# 7. Ejercicio 4 (productor-consumidor)

---

- ❖ Comprueba que la capacidad de almacenamiento es de 5 números
- ❖ Completa el código que falta en la clase para que los consumidores puedan retirar los datos
  - ❖ Los datos no podrán ser retirados si el acceso a los mismos está bloqueado
  - ❖ Los datos no podrán ser retirados si no hay datos que retirar
  - ❖ Se deberá generar un hueco libre al retirar los datos
- ❖ Ejecuta el programa y observa los resultados
- ❖ Modifica el bucle de extracción de datos por parte de los consumidores para que sólo consuman 10 elementos cada uno
- ❖ Ejecuta de nuevo y reflexiona sobre lo que ocurre con los productores pasado un tiempo